

COAL 8088 NASM Complete Code Cheat Sheet

BASIC STRUCTURE

```
nasm

; Every program ends with:
mov ax, 4c00h
int 21h
```

BITWISE OPERATIONS

1. Check Even/Odd

```
nasm

mov ax, 15      ; number to check
and ax, 1       ; AND with 1
cmp ax, 0
je even_num     ; if 0, even
; odd code
jmp done
even_num:
; even code
done:
mov ax, 4c00h
int 21h
```

2. Swap Two Numbers (XOR method)

```
nasm

mov ax, 10
mov bx, 20
xor ax, bx
xor bx, ax
xor ax, bx
; now ax=20, bx=10
mov ax, 4c00h
int 21h
```

3. Count Set Bits (Count 1s)

```
nasm

mov ax, 13      ; number
mov cx, 0       ; counter
mov dx, 16      ; 16 bits

count_loop:
shl ax, 1       ; shift left
jnc skip        ; if no carry, skip
inc cx
skip:
dec dx
jnz count_loop
; cx has count
mov ax, 4c00h
int 21h
```

4. Check if Power of 2

```
nasm

mov ax, 16      ; number
mov bx, ax
dec bx
and ax, bx
cmp ax, 0
je is_power     ; if 0, power of 2
; not power of 2
jmp done
is_power:
; is power of 2
done:
mov ax, 4c00h
int 21h
```

5. Set Specific Bit (Set bit 3)

```
nasm
```

```
mov ax, 10      ; number
or ax, 8        ; OR with 00001000b
; bit 3 is now set
mov ax, 4c00h
int 21h
```

6. Clear Specific Bit (Clear bit 3)

```
nasm

mov ax, 15      ; number
and ax, 0F7h    ; AND with 11110111b
; bit 3 is now cleared
mov ax, 4c00h
int 21h
```

7. Toggle Specific Bit (Toggle bit 3)

```
nasm

mov ax, 10      ; number
xor ax, 8       ; XOR with 00001000b
; bit 3 is toggled
mov ax, 4c00h
int 21h
```

8. Extract Bits (Extract bits 7-4)

```
nasm

mov ax, 0ABCDh  ; number
and ax, 0F0h    ; mask bits 7-4
mov cl, 4
shr ax, cl      ; shift right 4 times
; ax has extracted bits
mov ax, 4c00h
int 21h
```

9. Encrypt/Decrypt with Shift

```
nasm
```

```
; Encrypt (left shift 8)
```

```
mov ax, 1234h
```

```
mov cl, 8
```

```
shl ax, cl
```

```
; encrypted in ax
```

```
; Decrypt (right shift 8)
```

```
shr ax, cl
```

```
; decrypted in ax
```

```
mov ax, 4c00h
```

```
int 21h
```

10. Check Bit Position

```
nasm
```

```
mov ax, 15      ; number
```

```
mov cl, 3       ; bit position
```

```
mov bx, 1
```

```
shl bx, cl      ; create mask
```

```
and ax, bx
```

```
cmp ax, 0
```

```
je bit_clear    ; bit is 0
```

```
; bit is 1
```

```
jmp done
```

```
bit_clear:
```

```
; bit is 0
```

```
done:
```

```
mov ax, 4c00h
```

```
int 21h
```

1234 ARRAY OPERATIONS

1. Find Maximum

```
nasm
```

```
mov ax, [0]    ; first element
mov si, 2      ; start index
mov cx, 4      ; remaining elements
```

```
find_max:
cmp cx, 0
je max_done
mov bx, [si]
cmp bx, ax
jle skip_max
mov ax, bx
skip_max:
add si, 2
dec cx
jmp find_max
```

```
max_done:
; ax has max
mov ax, 4c00h
int 21h
```

2. Find Minimum

```
nasm
```

```
mov ax, [0]      ; first element
```

```
mov si, 2
```

```
mov cx, 4
```

```
find_min:
```

```
cmp cx, 0
```

```
je min_done
```

```
mov bx, [si]
```

```
cmp bx, ax
```

```
jge skip_min
```

```
mov ax, bx
```

```
skip_min:
```

```
add si, 2
```

```
dec cx
```

```
jmp find_min
```

```
min_done:
```

```
; ax has min
```

```
mov ax, 4c00h
```

```
int 21h
```

3. Sum of Array

```
nasm
```

```
mov ax, 0      ; sum
```

```
mov si, 0
```

```
mov cx, 5      ; count
```

```
sum_loop:
```

```
add ax, [si]
```

```
add si, 2
```

```
loop sum_loop
```

```
; ax has sum
```

```
mov ax, 4c00h
```

```
int 21h
```

4. Count Elements > Value

```
nasm
```

```
mov dx, 100    ; threshold value
mov si, 0
mov cx, 5      ; array size
mov bx, 0      ; counter
```

```
count_loop:
mov ax, [si]
cmp ax, dx
jle skip_count
inc bx
skip_count:
add si, 2
loop count_loop
; bx has count
mov ax, 4c00h
int 21h
```

5. Search Element

```
nasm

mov dx, 25    ; element to search
mov si, 0
mov cx, 5

search_loop:
mov ax, [si]
cmp ax, dx
je found
add si, 2
loop search_loop
; not found
jmp done

found:
; found at position si
done:
mov ax, 4c00h
int 21h
```

6. Reverse Array

```
nasm
```

```
mov si, 0      ; start
mov di, 8      ; end (5 elements)
```

```
reverse_loop:
```

```
cmp si, di
```

```
jge rev_done
```

```
mov ax, [si]
```

```
mov bx, [di]
```

```
mov [si], bx
```

```
mov [di], ax
```

```
add si, 2
```

```
sub di, 2
```

```
jmp reverse_loop
```

```
rev_done:
```

```
mov ax, 4c00h
```

```
int 21h
```

7. Copy Array

```
nasm
```

```
mov si, 0      ; source
```

```
mov di, 20     ; destination
```

```
mov cx, 5
```

```
copy_loop:
```

```
mov ax, [si]
```

```
mov [di], ax
```

```
add si, 2
```

```
add di, 2
```

```
loop copy_loop
```

```
mov ax, 4c00h
```

```
int 21h
```

8. Find Second Maximum

```
nasm
```



```

mov ax, [0]      ; first max
mov bx, [2]      ; second max
cmp ax, bx
jg order_ok
xchg ax, bx      ; swap if needed
order_ok:

mov si, 4
mov cx, 3        ; remaining elements

find_second:
cmp cx, 0
je done
mov dx, [si]
cmp dx, ax
jle check_second
mov bx, ax        ; shift down
mov ax, dx        ; new max
jmp next

check_second:
cmp dx, bx
jle next
mov bx, dx        ; new second max

next:
add si, 2
dec cx
jmp find_second

done:
; bx has second max
mov ax, 4c00h
int 21h

```

SORTING ALGORITHMS

1. Bubble Sort

nasm

```
mov si, 0      ; outer loop
```

```
outer:
```

```
cmp si, 4      ; n-1 passes
```

```
je sort_done
```

```
mov di, 0
```

```
inner:
```

```
mov cx, 4
```

```
sub cx, si
```

```
cmp di, cx
```

```
je next_outer
```

```
mov bx, di
```

```
mov ax, [bx]
```

```
add bx, 2
```

```
mov dx, [bx]
```

```
cmp ax, dx
```

```
jle no_swap
```

```
mov [bx-2], dx
```

```
mov [bx], ax
```

```
no_swap:
```

```
add di, 2
```

```
jmp inner
```

```
next_outer:
```

```
inc si
```

```
jmp outer
```

```
sort_done:
```

```
mov ax, 4c00h
```

```
int 21h
```

2. Selection Sort

```
nasm
```

```
mov si, 0      ; current position
```

```
outer_sel:
```

```
cmp si, 8      ; n-1
```

```
je sel_done
```

```
mov di, si
```

```
add di, 2      ; next position
```

```
mov bx, si     ; min index
```

```
mov ax, [si]   ; min value
```

```
inner_sel:
```

```
cmp di, 10     ; array end
```

```
je swap_min
```

```
mov dx, [di]
```

```
cmp dx, ax
```

```
jge skip_update
```

```
mov ax, dx
```

```
mov bx, di
```

```
skip_update:
```

```
add di, 2
```

```
jmp inner_sel
```

```
swap_min:
```

```
cmp bx, si
```

```
je next_pos
```

```
mov dx, [si]
```

```
mov [si], ax
```

```
mov [bx], dx
```

```
next_pos:
```

```
add si, 2
```

```
jmp outer_sel
```

```
sel_done:
```

```
mov ax, 4c00h
```

```
int 21h
```

3. Insertion Sort

```
nasm
```

```
mov si, 2      ; start from 2nd element
```

```
outer_ins:
```

```
cmp si, 10     ; array end
```

```
je ins_done
```

```
mov ax, [si]   ; current element
```

```
mov di, si
```

```
inner_ins:
```

```
cmp di, 0
```

```
je insert_pos
```

```
mov bx, di
```

```
sub bx, 2
```

```
mov dx, [bx]
```

```
cmp dx, ax
```

```
jle insert_pos
```

```
mov [di], dx   ; shift right
```

```
sub di, 2
```

```
jmp inner_ins
```

```
insert_pos:
```

```
mov [di], ax
```

```
add si, 2
```

```
jmp outer_ins
```

```
ins_done:
```

```
mov ax, 4c00h
```

```
int 21h
```

STRING OPERATIONS

1. String Length

```
nasm
```

```
mov si, 0      ; string address
mov cx, 0

len_loop:
mov al, [si]
cmp al, 0      ; null terminator
je len_done
inc cx
inc si
jmp len_loop

len_done:
; cx has length
mov ax, 4c00h
int 21h
```

2. String Copy

```
nasm

mov si, 0      ; source
mov di, 50     ; destination

copy_str:
mov al, [si]
mov [di], al
cmp al, 0
je copy_done
inc si
inc di
jmp copy_str

copy_done:
mov ax, 4c00h
int 21h
```

3. String Compare

```
nasm
```

```
mov si, 0      ; string 1
mov di, 50     ; string 2
```

```
cmp_loop:
mov al, [si]
mov bl, [di]
cmp al, bl
jne not_equal
cmp al, 0
je strings_equal
inc si
inc di
jmp cmp_loop
```

```
not_equal:
; strings not equal
jmp done
```

```
strings_equal:
; strings equal
done:
mov ax, 4c00h
int 21h
```

4. String Reverse

```
nasm
```

```
; Find length first
```

```
mov si, 0
```

```
mov cx, 0
```

```
len_loop:
```

```
mov al, [si]
```

```
cmp al, 0
```

```
je rev_start
```

```
inc cx
```

```
inc si
```

```
jmp len_loop
```

```
rev_start:
```

```
mov si, 0 ; start
```

```
dec si
```

```
add si, cx ; end position
```

```
mov di, 0
```

```
rev_loop:
```

```
cmp di, si
```

```
jge rev_done
```

```
mov al, [di]
```

```
mov bl, [si]
```

```
mov [di], bl
```

```
mov [si], al
```

```
inc di
```

```
dec si
```

```
jmp rev_loop
```

```
rev_done:
```

```
mov ax, 4c00h
```

```
int 21h
```

5. Count Character in String

```
nasm
```

```
mov si, 0      ; string
mov dl, 'a'    ; char to count
mov cx, 0      ; counter
```

```
count_char:
mov al, [si]
cmp al, 0
je count_done
cmp al, dl
jne skip_inc
inc cx
skip_inc:
inc si
jmp count_char
```

```
count_done:
; cx has count
mov ax, 4c00h
int 21h
```

6. Convert to Upper Case

```
nasm
```



```
mov si, 0

upper_loop:
mov al, [si]
cmp al, 0
je upper_done
cmp al, 'a'
jl skip_upper
cmp al, 'z'
jg skip_upper
sub al, 32      ; convert to upper
mov [si], al
skip_upper:
inc si
jmp upper_loop

upper_done:
mov ax, 4c00h
int 21h
```

7. Convert to Lower Case

```
nasm
```

```
mov si, 0

lower_loop:
mov al, [si]
cmp al, 0
je lower_done
cmp al, 'A'
jl skip_lower
cmp al, 'Z'
jg skip_lower
add al, 32      ; convert to lower
mov [si], al
skip_lower:
inc si
jmp lower_loop

lower_done:
mov ax, 4c00h
int 21h
```

8. Proper Case (First Letter Capital)

```
nasm
```

```
mov si, 0
mov bl, 1      ; flag for new word
```

```
proper_loop:
mov al, [si]
cmp al, 0
je proper_done
```

```
cmp al, ''
je space_found
cmp bl, 1
jne check_lower
; capitalize
cmp al, 'a'
jl not_lower
cmp al, 'z'
jg not_lower
sub al, 32
mov [si], al
mov bl, 0
jmp next_char
```

```
check_lower:
cmp al, 'A'
jl next_char
cmp al, 'Z'
jg next_char
add al, 32
mov [si], al
jmp next_char
```

```
space_found:
mov bl, 1
jmp next_char
```

```
not_lower:
mov bl, 0
```

```
next_char:
inc si
jmp proper_loop
```

```
proper_done:
```

```
mov ax, 4c00h
int 21h
```

12 ARITHMETIC OPERATIONS

1. Factorial

```
nasm

mov ax, 5      ; number
mov bx, 1      ; result
mov cx, ax

fact_loop:
cmp cx, 1
je fact_done
mul cx
dec cx
jmp fact_loop

fact_done:
; ax has factorial
mov ax, 4c00h
int 21h
```

2. Fibonacci Series

```
nasm

mov ax, 0      ; first
mov bx, 1      ; second
mov cx, 10     ; count

fib_loop:
mov dx, ax
add dx, bx
mov ax, bx
mov bx, dx
loop fib_loop
; bx has nth fibonacci
mov ax, 4c00h
int 21h
```

3. GCD (Greatest Common Divisor)

nasm

```
mov ax, 48      ; first number
mov bx, 18      ; second number

gcd_loop:
cmp bx, 0
je gcd_done
mov dx, 0
div bx          ; ax = ax/bx, dx = remainder
mov ax, bx
mov bx, dx
jmp gcd_loop

gcd_done:
; ax has GCD
mov ax, 4c00h
int 21h
```

4. LCM (Least Common Multiple)

nasm

```
mov ax, 12    ; first
mov bx, 18    ; second
mov cx, ax    ; save first
mov dx, bx    ; save second
```

```
; Find GCD first
```

```
gcd_loop:
cmp bx, 0
je calc_lcm
push dx
mov dx, 0
div bx
mov ax, bx
pop bx
mov bx, dx
jmp gcd_loop
```

```
calc_lcm:
```

```
;  $LCM = (num1 * num2) / GCD$ 
```

```
mov bx, ax    ; GCD in bx
```

```
mov ax, cx
```

```
mul dx
```

```
div bx
```

```
; ax has LCM
```

```
mov ax, 4c00h
```

```
int 21h
```

5. Check Prime Number

```
nasm
```

```
mov ax, 17      ; number to check
cmp ax, 2
jl not_prime
je is_prime
mov bx, 2       ; divisor
```

```
prime_loop:
mov dx, 0
push ax
div bx
cmp dx, 0
je not_prime
pop ax
inc bx
mov cx, bx
mul cx
cmp bx, ax
pop ax
jl prime_loop
```

```
is_prime:
; number is prime
jmp done
```

```
not_prime:
; not prime
done:
mov ax, 4c00h
int 21h
```

6. Sum of Digits

```
nasm
```

```

mov ax, 1234    ; number
mov bx, 0       ; sum
mov cx, 10      ; divisor

digit_loop:
cmp ax, 0
je sum_done
mov dx, 0
div cx          ; ax = quotient, dx = remainder
add bx, dx      ; add digit to sum
jmp digit_loop

sum_done:
; bx has sum
mov ax, 4c00h
int 21h

```

7. Reverse Number

```

nasm

mov ax, 1234    ; number
mov bx, 0       ; result
mov cx, 10

rev_num:
cmp ax, 0
je num_done
mov dx, 0
div cx          ; get last digit
push ax
mov ax, bx
mul cx
add ax, dx      ; result = result*10 + digit
mov bx, ax
pop ax
jmp rev_num

num_done:
; bx has reversed number
mov ax, 4c00h
int 21h

```


8. Check Palindrome Number

```
nasm

mov ax, 121    ; number
mov bx, ax     ; save original
mov cx, 0      ; reversed
mov dx, 10

pal_loop:
cmp ax, 0
je check_pal
push dx
mov dx, 0
div dx        ; get last digit
pop dx
push ax
mov ax, cx
mul dx
add ax, dx
mov cx, ax
pop ax
jmp pal_loop

check_pal:
cmp bx, cx
je palindrome
; not palindrome
jmp done

palindrome:
; is palindrome
done:
mov ax, 4c00h
int 21h
```

DISPLAY OPERATIONS

1. Print Single Character

```
nasm
```

```
mov dl, 'A'    ; character
mov ah, 02h
int 21h

mov ax, 4c00h
int 21h
```

2. Print String (Using Loop)

```
nasm

mov si, 0      ; string address

print_loop:
mov dl, [si]
cmp dl, 0
je print_done
mov ah, 02h
int 21h
inc si
jmp print_loop

print_done:
mov ax, 4c00h
int 21h
```

3. Print Number (Single Digit)

```
nasm

mov al, 7      ; digit
add al, 30h    ; convert to ASCII
mov dl, al
mov ah, 02h
int 21h

mov ax, 4c00h
int 21h
```

4. Print Multi-Digit Number

```
nasm
```

```

mov ax, 123    ; number
mov cx, 0      ; digit counter
mov bx, 10

push_digits:
mov dx, 0
div bx         ; divide by 10
push dx        ; save remainder
inc cx
cmp ax, 0
jne push_digits

print_digits:
pop dx
add dl, 30h    ; convert to ASCII
mov ah, 02h
int 21h
loop print_digits

mov ax, 4c00h
int 21h

```

5. New Line

```

nasm

mov dl, 0Dh    ; carriage return
mov ah, 02h
int 21h

mov dl, 0Ah    ; line feed
mov ah, 02h
int 21h

mov ax, 4c00h
int 21h

```

6. Print Pattern (Stars)

```

nasm

```

```
mov cx, 5      ; rows
```

```
row_loop:
```

```
push cx
```

```
mov dx, cx
```

```
star_loop:
```

```
mov ah, 02h
```

```
mov dl, '*'
```

```
int 21h
```

```
dec dx
```

```
jnz star_loop
```

```
mov dl, 0Dh    ; newline
```

```
mov ah, 02h
```

```
int 21h
```

```
mov dl, 0Ah
```

```
int 21h
```

```
pop cx
```

```
loop row_loop
```

```
mov ax, 4c00h
```

```
int 21h
```

SUBROUTINES (PROCEDURES)

1. Simple Subroutine Template

```
nasm
```

```
; Main program
start:
mov ax, 10
mov bx, 20
call add_nums    ; call subroutine
; result in cx

mov ax, 4c00h
int 21h

; Subroutine
add_nums:
push ax          ; save registers
push bx
mov cx, ax
add cx, bx
pop bx           ; restore registers
pop ax
ret
```

2. Subroutine with Parameters

```
nasm
```

; Main

start:

mov ax, 15 ; parameter 1

mov bx, 25 ; parameter 2

call multiply

; result in cx

mov ax, 4c00h

int 21h

; Subroutine

multiply:

push ax

push bx

push dx

mov cx, 0

mov dx, bx

mul_loop:

add cx, ax

dec dx

jnz mul_loop

pop dx

pop bx

pop ax

ret

3. Print Character Subroutine

nasm

```
start:
mov dl, 'A'
call print_char
mov dl, 'B'
call print_char
```

```
mov ax, 4c00h
int 21h
```

```
print_char:
push ax
mov ah, 02h
int 21h
pop ax
ret
```

4. Recursive Factorial

```
nasm
```

```
start:
mov ax, 5
call factorial
; result in ax
```

```
mov ax, 4c00h
int 21h
```

```
factorial:
cmp ax, 1
je base_case
push ax
dec ax
call factorial
pop bx
mul bx
ret
```

```
base_case:
mov ax, 1
ret
```

1. Print X Pattern (Cross)

nasm


```
mov cx, 10      ; size
```

```
mov si, 0       ; line
```

```
next_line:
```

```
mov di, 0       ; column
```

```
check_col:
```

```
cmp di, 10
```

```
je line_done
```

```
mov ax, di
```

```
cmp ax, si      ; left diagonal
```

```
je print_x
```

```
mov ax, 9
```

```
sub ax, si
```

```
cmp ax, di      ; right diagonal
```

```
je print_x
```

```
mov dl, ''
```

```
jmp print_it
```

```
print_x:
```

```
mov dl, 'X'
```

```
print_it:
```

```
mov ah, 02h
```

```
int 21h
```

```
inc di
```

```
jmp check_col
```

```
line_done:
```

```
mov dl, 0Dh
```

```
mov ah, 02h
```

```
int 21h
```

```
mov dl, 0Ah
```

```
int 21h
```

```
inc si
```

```
dec cx
```

```
jnz next_line
```

```
mov ax, 4c00h
int 21h
```

2. Print Triangle

```
nasm

mov cx, 5      ; rows

tri_row:
push cx
mov dx, cx

tri_col:
mov ah, 02h
mov dl, '*'
int 21h
dec dx
jnz tri_col

mov dl, 0Dh
mov ah, 02h
int 21h
mov dl, 0Ah
int 21h

pop cx
loop tri_row

mov ax, 4c00h
int 21h
```

3. Print Pyramid

```
nasm
```

```
mov cx, 5      ; rows
```

```
mov si, 1
```

```
pyr_row:
```

```
push cx
```

```
mov dx, cx
```

```
dec dx
```

```
; Print spaces
```

```
space_loop:
```

```
cmp dx, 0
```

```
je print_stars
```

```
mov ah, 02h
```

```
mov dl, ' '
```

```
int 21h
```

```
pop dx
```

```
push dx
```

```
dec dx
```

```
jmp space_loop
```

```
print_stars:
```

```
mov dx, si
```

```
star_loop:
```

```
mov ah, 02h
```

```
mov dl, '*'
```

```
int 21h
```

```
dec dx
```

```
jnz star_loop
```

```
mov dl, 0Dh
```

```
mov ah, 02h
```

```
int 21h
```

```
mov dl, 0Ah
```

```
int 21h
```

```
add si, 2
```

```
pop cx
```

```
loop pyr_row
```

```
mov ax, 4c00h
```

```
int 21h
```



DATA EXTRACTION (BIT MANIPULATION)

1. Extract Transaction Data

nasm

; 16-bit format:
; Bit 15: Sign (0=deposit, 1=withdrawal)
; Bits 14-8: Transaction ID
; Bits 7-0: Amount

mov ax, [si] ; get transaction

; Decrypt (if encrypted with shift)

mov cl, 8

shr ax, cl

; Extract sign bit

mov dx, ax

and dx, 8000h

shr dx, 15 ; sign in DX (0 or 1)

; Extract transaction ID

mov bx, ax

and bx, 7F00h ; mask bits 14-8

mov cl, 8

shr bx, cl ; ID in BX

; Extract amount

and ax, 00FFh ; amount in AX

; If withdrawal, get absolute

cmp dx, 1

jne positive

neg ax

and ax, 00FFh

positive:

; Check if > 100

cmp ax, 100

jle no_overflow

inc cx ; increment overflow counter

no_overflow:

mov ax, 4c00h

int 21h

2. Pack Data into Bits

```
nasm

; Pack: sign(1 bit) + id(7 bits) + amount(8 bits)
mov al, 1      ; sign bit
mov bl, 45     ; transaction ID
mov cl, 150    ; amount

; Combine
shl ax, 7      ; make room for ID
or al, bl      ; add ID
shl ax, 8      ; make room for amount
or al, cl      ; add amount

; Encrypt (shift left 8)
mov cl, 8
shl ax, cl

mov ax, 4c00h
int 21h
```

SEARCHING ALGORITHMS

1. Linear Search

```
nasm
```

```
mov dx, 25      ; search value
```

```
mov si, 0
```

```
mov cx, 5      ; array size
```

```
search:
```

```
mov ax, [si]
```

```
cmp ax, dx
```

```
je found
```

```
add si, 2
```

```
loop search
```

```
; not found
```

```
jmp done
```

```
found:
```

```
; found at position SI
```

```
done:
```

```
mov ax, 4c00h
```

```
int 21h
```

2. Binary Search (Sorted Array)

```
nasm
```

```
mov dx, 25      ; search value
mov si, 0       ; left
mov di, 8       ; right (last index)

bin_search:
cmp si, di
jg not_found

mov ax, si
add ax, di
shr ax, 1       ; mid = (left+right)/2
and ax, 0FFFEh  ; align to even (word boundary)
mov bx, ax

mov cx, [bx]    ; array[mid]
cmp cx, dx
je found
jl search_right

; Search left
mov di, bx
sub di, 2
jmp bin_search

search_right:
mov si, bx
add si, 2
jmp bin_search

found:
; found at BX
jmp done

not_found:
; not found

done:
mov ax, 4c00h
int 21h
```


1. Count Elements Meeting Condition

```
nasm

; Count elements > 100
mov si, 0
mov cx, 5      ; array size
mov bx, 0      ; counter
mov dx, 100    ; threshold

count_loop:
mov ax, [si]
cmp ax, dx
jle skip
inc bx
skip:
add si, 2
loop count_loop
; BX has count
mov ax, 4c00h
int 21h
```

2. Find Average

```
nasm

mov ax, 0      ; sum
mov si, 0
mov cx, 5

sum_array:
add ax, [si]
add si, 2
loop sum_array

mov bx, 5      ; divisor
mov dx, 0
div bx
; AX has average
mov ax, 4c00h
int 21h
```

3. Separate Positive/Negative

```
nasm

mov si, 0      ; source
mov di, 50     ; positive array
mov bp, 100    ; negative array
mov cx, 5

separate:
mov ax, [si]
test ax, 8000h ; check sign bit
jnz negative
mov [di], ax
add di, 2
jmp next

negative:
mov [bp], ax
add bp, 2

next:
add si, 2
loop separate

mov ax, 4c00h
int 21h
```

IMPORTANT NOTES

Register Usage:

- **AX:** Accumulator (arithmetic, I/O)
- **BX:** Base (indexing, pointer)
- **CX:** Counter (loops)
- **DX:** Data (arithmetic, I/O)
- **SI:** Source Index
- **DI:** Destination Index
- **BP:** Base Pointer

- **SP:** Stack Pointer

Common Instructions:

- **MOV:** Move data
- **ADD/SUB:** Arithmetic
- **MUL/DIV:** Multiply/Divide
- **AND/OR/XOR:** Bitwise operations
- **SHL/SHR:** Shift left/right
- **CMP:** Compare
- **JE/JNE/JG/JL:** Conditional jumps
- **LOOP:** Loop with CX counter
- **PUSH/POP:** Stack operations
- **CALL/RET:** Subroutine calls

Flags:

- **ZF:** Zero Flag
- **CF:** Carry Flag
- **SF:** Sign Flag
- **OF:** Overflow Flag

DOS Interrupts:

- **INT 21h, AH=02h:** Print character (DL)
- **INT 21h, AH=09h:** Print string (DS:DX)
- **INT 21h, AH=4Ch:** Exit program
- **INT 10h:** Video services

Tips:

1. Always initialize registers
2. Use PUSH/POP to save registers
3. Remember word = 2 bytes

4. Check array boundaries
 5. Handle special cases (empty, single element)
 6. Comment your code
 7. Test with simple values first
-

QUICK REFERENCE

End Program:

```
nasm  
  
mov ax, 4c00h  
int 21h
```

Print Character:

```
nasm  
  
mov dl, 'A'  
mov ah, 02h  
int 21h
```

Newline:

```
nasm  
  
mov dl, 0Dh  
mov ah, 02h  
int 21h  
mov dl, 0Ah  
int 21h
```

Loop Template:

```
nasm  
  
mov cx, 10  
loop_start:  
; your code  
loop loop_start
```

Array Access (Word):

```
nasm  
  
mov si, 0      ; index 0  
mov ax, [si]   ; get element  
add si, 2      ; next element
```

MATRIX OPERATIONS

1. 2D Array Access (3x3 matrix)

```
nasm  
  
; Access element [row][col]  
; Formula: address = base + (row * cols + col) * 2  
mov ax, 1      ; row  
mov bx, 3      ; columns per row  
mul bx  
add ax, 2      ; column  
mov si, ax  
shl si, 1      ; multiply by 2 (word size)  
mov dx, [si]   ; get element  
  
mov ax, 4c00h  
int 21h
```

2. Sum of Matrix Row

```
nasm
```

```

; Sum row 1 (3 columns)
mov ax, 1      ; row number
mov bx, 3
mul bx
shl ax, 1      ; convert to byte offset
mov si, ax

mov cx, 3      ; columns
mov dx, 0      ; sum

row_sum:
mov bx, [si]
add dx, bx
add si, 2
loop row_sum
; DX has row sum

mov ax, 4c00h
int 21h

```

3. Sum of Matrix Column

```

nasm

; Sum column 1 (3 rows)
mov si, 2      ; column 1 offset
mov cx, 3      ; rows
mov dx, 0      ; sum

col_sum:
mov ax, [si]
add dx, ax
add si, 6      ; jump to next row (3 cols * 2 bytes)
loop col_sum
; DX has column sum

mov ax, 4c00h
int 21h

```

4. Transpose Matrix (3x3)

```

nasm

```

```
; Swap elements across diagonal
```

```
mov si, 0
```

```
mov cx, 3      ; rows
```

```
trans_row:
```

```
push cx
```

```
mov di, si
```

```
add di, 6      ; next row start
```

```
trans_col:
```

```
cmp di, 18     ; end of matrix
```

```
jge next_row
```

```
mov ax, [si+2]  ; get [i][j]
```

```
mov bx, [di]    ; get [j][i]
```

```
mov [si+2], bx  ; swap
```

```
mov [di], ax
```

```
add di, 6
```

```
add si, 2
```

```
pop cx
```

```
push cx
```

```
loop trans_col
```

```
next_row:
```

```
pop cx
```

```
add si, 6
```

```
loop trans_row
```

```
mov ax, 4c00h
```

```
int 21h
```

5. Matrix Diagonal Sum

```
nasm
```

```
; Sum main diagonal (3x3)
```

```
mov si, 0
```

```
mov cx, 3
```

```
mov dx, 0
```

```
diag_sum:
```

```
mov ax, [si]
```

```
add dx, ax
```

```
add si, 8      ; next diagonal (3*2 + 2)
```

```
loop diag_sum
```

```
; DX has diagonal sum
```

```
mov ax, 4c00h
```

```
int 21h
```

NUMBER SYSTEM CONVERSIONS

1. Decimal to Binary (Display)

```
nasm
```



```
mov ax, 13      ; decimal number
```

```
mov cx, 16      ; 16 bits
```

```
bin_loop:
```

```
shl ax, 1       ; shift left, MSB to carry
```

```
jc print_one
```

```
mov dl, '0'
```

```
jmp print_bit
```

```
print_one:
```

```
mov dl, '1'
```

```
print_bit:
```

```
push ax
```

```
mov ah, 02h
```

```
int 21h
```

```
pop ax
```

```
loop bin_loop
```

```
mov ax, 4c00h
```

```
int 21h
```

2. Binary to Decimal

```
nasm
```

```
; Binary string "1101" to decimal
mov si, 0      ; string position
mov bx, 0      ; result
mov cx, 2      ; multiplier (power of 2)
```

```
bin_to_dec:
mov al, [si]
cmp al, 0
je conv_done
sub al, '0'    ; convert ASCII to digit
mov dx, 0
mov dl, al
push cx
mov ax, bx
mul cx
add ax, dx
mov bx, ax
pop cx
shl cx, 1      ; multiply by 2
inc si
jmp bin_to_dec
```

```
conv_done:
; BX has decimal value
mov ax, 4c00h
int 21h
```

3. Decimal to Hexadecimal (Display)

```
nasm
```

```
mov ax, 255    ; decimal number
```

```
mov cx, 4      ; 4 hex digits
```

```
hex_loop:
```

```
push cx
```

```
mov cx, 4
```

```
rol ax, cl     ; rotate left 4 bits
```

```
mov bx, ax
```

```
and bx, 0Fh    ; get last 4 bits
```

```
cmp bx, 9
```

```
jg hex_letter
```

```
add bl, '0'
```

```
jmp print_hex
```

```
hex_letter:
```

```
add bl, 37h    ; 'A' - 10
```

```
print_hex:
```

```
push ax
```

```
mov dl, bl
```

```
mov ah, 02h
```

```
int 21h
```

```
pop ax
```

```
pop cx
```

```
loop hex_loop
```

```
mov ax, 4c00h
```

```
int 21h
```

4. Hexadecimal to Decimal

```
nasm
```

```
; Hex string "FF" to decimal
```

```
mov si, 0
```

```
mov bx, 0      ; result
```

```
hex_to_dec:
```

```
mov al, [si]
```

```
cmp al, 0
```

```
je hex_done
```

```
cmp al, '9'
```

```
jle digit
```

```
sub al, 7      ; for A-F
```

```
digit:
```

```
sub al, '0'
```

```
mov cl, 4
```

```
shl bx, cl     ; multiply by 16
```

```
mov dx, 0
```

```
mov dl, al
```

```
add bx, dx
```

```
inc si
```

```
jmp hex_to_dec
```

```
hex_done:
```

```
; BX has decimal
```

```
mov ax, 4c00h
```

```
int 21h
```

SUBSTRING & STRING MATCHING

1. Find Substring

```
nasm
```

```

; Find "CD" in "ABCDEF"
mov si, 0      ; main string
mov di, 50     ; substring

search_sub:
mov al, [si]
cmp al, 0
je not_found_sub

mov bx, si
mov dx, di

check_match:
mov al, [bx]
mov cl, [dx]
cmp cl, 0
je found_sub   ; end of substring = match
cmp al, cl
jne next_pos
inc bx
inc dx
jmp check_match

next_pos:
inc si
jmp search_sub

found_sub:
; Found at position SI
jmp done_sub

not_found_sub:
; Not found

done_sub:
mov ax, 4c00h
int 21h

```

2. Longest Common Substring

```
nasm
```

; Find longest matching substring in two strings

mov cx, 0 *; max length*

mov si, 0 *; string A position*

check_a:

mov al, [si]

cmp al, 0

je finish

mov di, 0 *; string B position*

check_b:

mov bl, [di]

cmp bl, 0

je next_a

cmp al, bl

jne next_b

; Match found, count length

mov bx, 0 *; length counter*

mov bp, si

mov dx, di

count_len:

push si

push di

mov si, bp

mov di, dx

mov al, [si]

mov bl, [di]

cmp al, bl

jne save_max

cmp al, 0

je save_max

inc bx

inc bp

inc dx

pop di

pop si

jmp count_len

save_max:

pop di

```
pop si
cmp bx, cx
jle next_b
mov cx, bx    ; update max
```

```
next_b:
inc di
jmp check_b
```

```
next_a:
inc si
jmp check_a
```

```
finish:
; CX has max length
mov ax, 4c00h
int 21h
```

3. Count Substring Occurrences

```
nasm
```

```
; Count how many times "AB" appears in string
```

```
mov si, 0      ; main string
```

```
mov cx, 0      ; counter
```

```
count_occur:
```

```
mov al, [si]
```

```
cmp al, 0
```

```
je count_done
```

```
mov bx, si
```

```
mov di, 50     ; substring
```

```
check_sub:
```

```
mov al, [bx]
```

```
mov dl, [di]
```

```
cmp dl, 0
```

```
je found_match
```

```
cmp al, dl
```

```
jne next_search
```

```
inc bx
```

```
inc di
```

```
jmp check_sub
```

```
found_match:
```

```
inc cx         ; increment counter
```

```
next_search:
```

```
inc si
```

```
jmp count_occur
```

```
count_done:
```

```
; CX has count
```

```
mov ax, 4c00h
```

```
int 21h
```

ROTATION OPERATIONS

1. Rotate Array Left

```
nasm
```



```
; Rotate array left by 1 position
mov ax, [0]      ; save first element
mov si, 0
mov cx, 4        ; n-1 elements
```

```
rotate_left:
mov bx, si
add bx, 2
mov dx, [bx]
mov [si], dx
add si, 2
loop rotate_left
```

```
mov [si], ax      ; place first at end
```

```
mov ax, 4c00h
int 21h
```

2. Rotate Array Right

```
nasm

; Rotate array right by 1 position
mov di, 8        ; last position (5 elements)
mov ax, [di]     ; save last element
mov cx, 4
```

```
rotate_right:
mov bx, di
sub bx, 2
mov dx, [bx]
mov [di], dx
mov di, bx
loop rotate_right
```

```
mov [0], ax      ; place last at start
```

```
mov ax, 4c00h
int 21h
```

3. Rotate Bits Left

```
nasm
```

```
mov ax, 0ABCDh ; number
mov cl, 4 ; rotate by 4 bits
rol ax, cl
; AX has rotated value
```

```
mov ax, 4c00h
int 21h
```

4. Rotate Bits Right

```
nasm

mov ax, 0ABCDh
mov cl, 4
ror ax, cl
; AX has rotated value
```

```
mov ax, 4c00h
int 21h
```

ADVANCED PATTERNS

1. Print Diamond Pattern

```
nasm
```

; Upper half

mov cx, 3 ; rows

mov si, 1 ; stars

upper_half:

push cx

mov dx, cx

dec dx

; Spaces

space1:

cmp dx, 0

je stars1

mov ah, 02h

mov dl, ' '

int 21h

dec dx

jmp space1

stars1:

mov dx, si

star_loop1:

mov ah, 02h

mov dl, '*'

int 21h

dec dx

jnz star_loop1

mov dl, 0Dh

mov ah, 02h

int 21h

mov dl, 0Ah

int 21h

add si, 2

pop cx

loop upper_half

; Lower half

mov cx, 2

sub si, 4

lower_half:

```
push cx
mov dx, cx

space2:
mov ah, 02h
mov dl, ' '
int 21h
dec dx
jnz space2

mov dx, si
star_loop2:
mov ah, 02h
mov dl, '*'
int 21h
dec dx
jnz star_loop2

mov dl, 0Dh
mov ah, 02h
int 21h
mov dl, 0Ah
int 21h

sub si, 2
pop cx
loop lower_half

mov ax, 4c00h
int 21h
```

2. Print Hollow Square

```
nasm
```

```
mov cx, 5      ; size
```

```
square_row:
```

```
push cx
```

```
mov dx, 5
```

```
square_col:
```

```
; Check if border
```

```
cmp cx, 5
```

```
je border
```

```
cmp cx, 1
```

```
je border
```

```
cmp dx, 5
```

```
je border
```

```
cmp dx, 1
```

```
je border
```

```
; Inside - print space
```

```
mov ah, 02h
```

```
mov bl, ' '
```

```
jmp print_sq
```

```
border:
```

```
mov bl, '*'
```

```
print_sq:
```

```
push dx
```

```
mov dl, bl
```

```
int 21h
```

```
pop dx
```

```
dec dx
```

```
jnz square_col
```

```
mov dl, 0Dh
```

```
mov ah, 02h
```

```
int 21h
```

```
mov dl, 0Ah
```

```
int 21h
```

```
pop cx
```

```
loop square_row
```

```
mov ax, 4c00h  
int 21h
```

3. Print Number Pattern

```
nasm
```

; Pattern:

; 1

; 1 2

; 1 2 3

; 1 2 3 4

mov **cx**, 4 *; rows*

mov **si**, 1

num_row:

push **cx**

mov **di**, 1 *; number to print*

num_col:

cmp **di**, **si**

jg row_done

push **di**

mov **dl**, **di**

add **dl**, '0'

mov **ah**, 02h

int 21h

mov **dl**, ''

int 21h

pop **di**

inc **di**

jmp num_col

row_done:

mov **dl**, 0Dh

mov **ah**, 02h

int 21h

mov **dl**, 0Ah

int 21h

inc **si**

pop **cx**

loop num_row

mov **ax**, 4c00h

int 21h

ADVANCED SEARCHING

1. Find All Occurrences

```
nasm

; Find all positions where element appears
mov dx, 25      ; search value
mov si, 0
mov di, 100     ; result array
mov cx, 5       ; array size
mov bx, 0       ; count

find_all:
mov ax, [si]
cmp ax, dx
jne skip_save
mov [di], si    ; save position
add di, 2
inc bx

skip_save:
add si, 2
loop find_all
; BX has count

mov ax, 4c00h
int 21h
```

2. Find Closest Value

```
nasm
```



```

; Find element closest to target
mov dx, 50      ; target
mov si, 0
mov ax, [si]    ; first element as closest
mov bx, ax
sub bx, dx
; Get absolute difference
cmp bx, 0
jge pos1
neg bx
pos1:
mov bp, bx      ; min difference
mov di, ax      ; closest value

find_closest:
add si, 2
cmp si, 10
je closest_done
mov ax, [si]
mov bx, ax
sub bx, dx
cmp bx, 0
jge pos2
neg bx
pos2:
cmp bx, bp
jge next_elem
mov bp, bx
mov di, ax

next_elem:
jmp find_closest

closest_done:
; DI has closest value
mov ax, 4c00h
int 21h

```

SPECIAL ALGORITHMS

1. Remove Duplicates

```
nasm

; Remove duplicates from sorted array
mov si, 0      ; read position
mov di, 0      ; write position
mov cx, 5      ; size

remove_dup:
mov ax, [si]
cmp si, 0
je write_first

mov bx, [di-2] ; previous unique element
cmp ax, bx
je skip_write

write_first:
mov [di], ax
add di, 2

skip_write:
add si, 2
loop remove_dup
; DI/2 = new size

mov ax, 4c00h
int 21h
```

2. Merge Two Sorted Arrays

```
nasm
```

; Merge array1 and array2 into result

```
mov si, 0      ; array1 index
mov di, 20     ; array2 index
mov bp, 50     ; result index
mov cx, 3      ; size of array1
mov dx, 3      ; size of array2
```

merge_loop:

```
cmp cx, 0
je copy_arr2
cmp dx, 0
je copy_arr1
```

```
mov ax, [si]
mov bx, [di]
cmp ax, bx
jg from_arr2
```

```
from_arr1:
mov [bp], ax
add si, 2
add bp, 2
dec cx
jmp merge_loop
```

```
from_arr2:
mov [bp], bx
add di, 2
add bp, 2
dec dx
jmp merge_loop
```

```
copy_arr1:
cmp cx, 0
je merge_done
mov ax, [si]
mov [bp], ax
add si, 2
add bp, 2
dec cx
jmp copy_arr1
```

copy_arr2:

```
cmp dx, 0
je merge_done
mov ax, [di]
mov [bp], ax
add di, 2
add bp, 2
dec dx
jmp copy_arr2
```

```
merge_done:
mov ax, 4c00h
int 21h
```

3. Partition Array (QuickSort Helper)

```
nasm
```

```
; Partition around pivot (last element)
mov di, 8      ; last position
mov dx, [di]   ; pivot
mov si, 0      ; smaller elements index
mov bx, 0      ; current index
```

```
partition_loop:
cmp bx, 8      ; reached pivot
je place_pivot
mov ax, [bx]
cmp ax, dx
jg skip_swap
```

```
; Swap with si position
push dx
mov dx, [si]
mov [bx], dx
mov [si], ax
pop dx
add si, 2
```

```
skip_swap:
add bx, 2
jmp partition_loop
```

```
place_pivot:
mov ax, [si]
mov [si], dx
mov [di], ax
; SI has pivot final position
```

```
mov ax, 4c00h
int 21h
```



VIDEO MEMORY & SCREEN OPERATIONS

1. Clear Screen

```
nasm
```

```
mov ax, 0600h    ; scroll up function
mov bh, 07h      ; attribute (white on black)
mov cx, 0        ; top left (row 0, col 0)
mov dx, 184Fh    ; bottom right (row 24, col 79)
int 10h

mov ax, 4c00h
int 21h
```

2. Set Cursor Position

```
nasm

mov ah, 02h      ; set cursor position
mov bh, 0        ; page number
mov dh, 10       ; row
mov dl, 20       ; column
int 10h

mov ax, 4c00h
int 21h
```

3. Write Character at Cursor (with color)

```
nasm

mov ah, 09h      ; write char with attribute
mov al, 'A'      ; character
mov bh, 0        ; page number
mov bl, 0Eh      ; color (yellow)
mov cx, 1        ; repeat count
int 10h

mov ax, 4c00h
int 21h
```

4. Write Directly to Video Memory

```
nasm
```

```

; Video memory starts at B800:0000
mov ax, 0B800h ; video memory segment
mov es, ax     ; set ES to video memory
mov di, 0      ; offset (top-left)

mov al, 'A'    ; character
mov ah, 0Eh    ; attribute (yellow)
mov es:[di], ax ; write to video memory

mov ax, 4c00h
int 21h

```

5. Write String to Video Memory

```

nasm

mov ax, 0B800h
mov es, ax
mov di, 0      ; screen position
mov si, 0      ; string address
mov ah, 0Fh    ; attribute (white on black)

write_loop:
mov al, [si]
cmp al, 0
je write_done
mov es:[di], ax ; write char + attribute
add di, 2      ; next screen position (char + attr = 2 bytes)
inc si
jmp write_loop

write_done:
mov ax, 4c00h
int 21h

```

6. Fill Screen with Character

```

nasm

```

```
mov ax, 0B800h
mov es, ax
mov di, 0
mov cx, 2000    ; 80*25 = 2000 characters
```

```
mov al, '*'      ; character
mov ah, 0Ch       ; attribute (light red)
```

```
fill_loop:
mov es:[di], ax
add di, 2
loop fill_loop
```

```
mov ax, 4c00h
int 21h
```

7. Print at Specific Position (Video Memory)

```
nasm

; Print 'X' at row 10, column 20
mov ax, 0B800h
mov es, ax

; Calculate offset: (row * 80 + col) * 2
mov ax, 10      ; row
mov bx, 80
mul bx          ; ax = row * 80
add ax, 20      ; add column
shl ax, 1       ; multiply by 2
mov di, ax

mov al, 'X'
mov ah, 0Eh     ; yellow
mov es:[di], ax

mov ax, 4c00h
int 21h
```

8. Draw Box/Border

```
nasm
```



```

mov ax, 0B800h
mov es, ax
mov ah, 0Fh    ; white

; Top border
mov di, 0
mov cx, 80
mov al, '-'
top_border:
mov es:[di], ax
add di, 2
loop top_border

; Bottom border
mov di, 3840    ; row 24 * 80 * 2
mov cx, 80
bot_border:
mov es:[di], ax
add di, 2
loop bot_border

mov ax, 4c00h
int 21h

```

9. Scroll Screen Up

```

nasm

mov ah, 06h    ; scroll up
mov al, 1      ; lines to scroll (0 = clear)
mov bh, 07h    ; attribute
mov ch, 0      ; top row
mov cl, 0      ; left column
mov dh, 24     ; bottom row
mov dl, 79     ; right column
int 10h

mov ax, 4c00h
int 21h

```

10. Change Background Color

```

nasm

```

```
mov ax, 0600h    ; clear screen
mov bh, 1Fh      ; attribute (white text on blue bg)
mov cx, 0
mov dx, 184Fh
int 10h

mov ax, 4c00h
int 21h
```

11. Print Colored Text at Position

```
nasm
```

; Print "HELLO" at row 5, col 10 in red

mov ah, 02h ; set cursor

mov bh, 0

mov dh, 5 ; row

mov dl, 10 ; column

int 10h

mov si, 0 ; string address

print_colored:

mov al, [si]

cmp al, 0

je print_done_col

mov ah, 09h ; write with attribute

mov bh, 0

mov bl, 0Ch ; red color

mov cx, 1

int 10h

; Move cursor right

mov ah, 02h

inc dl

int 10h

inc si

jmp print_colored

print_done_col:

mov ax, 4c00h

int 21h

12. Color Codes Reference

nasm

; Background colors (high nibble):
; 0 = Black, 1 = Blue, 2 = Green, 3 = Cyan
; 4 = Red, 5 = Magenta, 6 = Brown, 7 = Light Gray

; Foreground colors (low nibble):
; 0 = Black, 1 = Blue, 2 = Green, 3 = Cyan
; 4 = Red, 5 = Magenta, 6 = Brown, 7 = Light Gray
; 8 = Dark Gray, 9 = Light Blue, A = Light Green, B = Light Cyan
; C = Light Red, D = Light Magenta, E = Yellow, F = White

; Example: 0Eh = Yellow text on black background
; Example: 1Fh = White text on blue background
; Example: 4Eh = Yellow text on red background

13. Complete X Pattern with Video Memory

nasm

; Clear screen first

mov ax, 0600h

mov bh, 07h

mov cx, 0

mov dx, 184Fh

int 10h

; Setup video memory

mov ax, 0B800h

mov es, ax

mov cx, 10 ; size

mov si, 0 ; row

draw_x:

; Calculate positions

mov ax, si

mov bx, 80

mul bx ; row * 80

mov di, ax

shl di, 1 ; convert to byte offset

; Left diagonal

push di

add di, si

add di, si ; add column * 2

mov al, 'X'

mov ah, 0Eh ; yellow

mov es:[di], ax

pop di

; Right diagonal

mov ax, 9

sub ax, si ; 9 - row = column

add ax, ax ; multiply by 2

add di, ax

mov al, 'X'

mov ah, 0Ch ; red

mov es:[di], ax

inc si

dec cx

jnz draw_x

```
mov ax, 4c00h
```

```
int 21h
```

14. Print Pattern with Colors

```
nasm
```

; Clear screen

mov ax, 0600h

mov bh, 07h

mov cx, 0

mov dx, 184Fh

int 10h

mov ax, 0B800h

mov es, ax

mov di, 0

mov cx, 5 *; rows*

mov bl, 0Ch *; start color (red)*

pattern_row:

push cx

mov dx, cx *; stars per row*

pattern_stars:

mov al, '*'

mov ah, bl *; current color*

mov es:[di], ax

add di, 2

dec dx

jnz pattern_stars

; Move to next row (80 chars per row)

push cx

mov cx, 80

sub cx, [sp+2] *; 80 - stars printed*

shl cx, 1 *; multiply by 2*

add di, cx

pop cx

inc bl *; change color*

pop cx

loop pattern_row

mov ax, 4c00h

int 21h

15. Blinking Text

```
nasm

mov ah, 09h
mov al, 'A'
mov bh, 0
mov bl, 8Fh    ; blink bit (bit 7) + white
mov cx, 1
int 10h

mov ax, 4c00h
int 21h
```



INPUT/OUTPUT OPERATIONS

1. Read Single Character

```
nasm

mov ah, 01h    ; read character
int 21h
; character in AL

mov ax, 4c00h
int 21h
```

2. Read String

```
nasm
```



```
mov si, 0      ; buffer address
```

```
read_str:
```

```
mov ah, 01h
```

```
int 21h
```

```
cmp al, 0Dh    ; Enter key
```

```
je str_done
```

```
mov [si], al
```

```
inc si
```

```
jmp read_str
```

```
str_done:
```

```
mov byte [si], 0 ; null terminate
```

```
mov ax, 4c00h
```

```
int 21h
```

3. Read Number

```
nasm
```

```
mov bx, 0      ; result
mov cx, 10     ; multiplier

read_num:
mov ah, 01h
int 21h
cmp al, 0Dh
je num_done
sub al, '0'    ; convert to digit
push ax
mov ax, bx
mul cx
mov bx, ax
pop ax
mov ah, 0
add bx, ax
jmp read_num

num_done:
; BX has number
mov ax, 4c00h
int 21h
```

EXAM PRO TIPS

Quick Checklist:

✓ Initialize all registers before use ✓ Save registers with PUSH before subroutine ✓ Restore with POP in reverse order ✓ Remember: WORD = 2 bytes, increment by 2 ✓ Always handle edge cases (empty, single element) ✓ Test boundary conditions ✓ Use meaningful labels (loop1, done, found) ✓ Add simple comments ✓ End with `mov ax, 4c00h` and `int 21h`

Common Mistakes to Avoid:

- ✗ Forgetting to increment SI/DI by 2 for words
- ✗ Not initializing counters (CX)
- ✗ Infinite loops (missing loop exit condition)
- ✗ Not saving/restoring registers in subroutines
- ✗ Wrong jump conditions (JE vs JNE, JG vs JL)
- ✗ Forgetting null terminator in strings
- ✗ Not handling divide by zero

Time-Saving Shortcuts:

- Use `loop` instruction with CX
 - Use `test` instead of `cmp` for bit checking
 - Use `xchg` for swapping
 - Use `lea` for address calculations
 - Combine operations where possible
-

GOOD LUCK! 🎓